

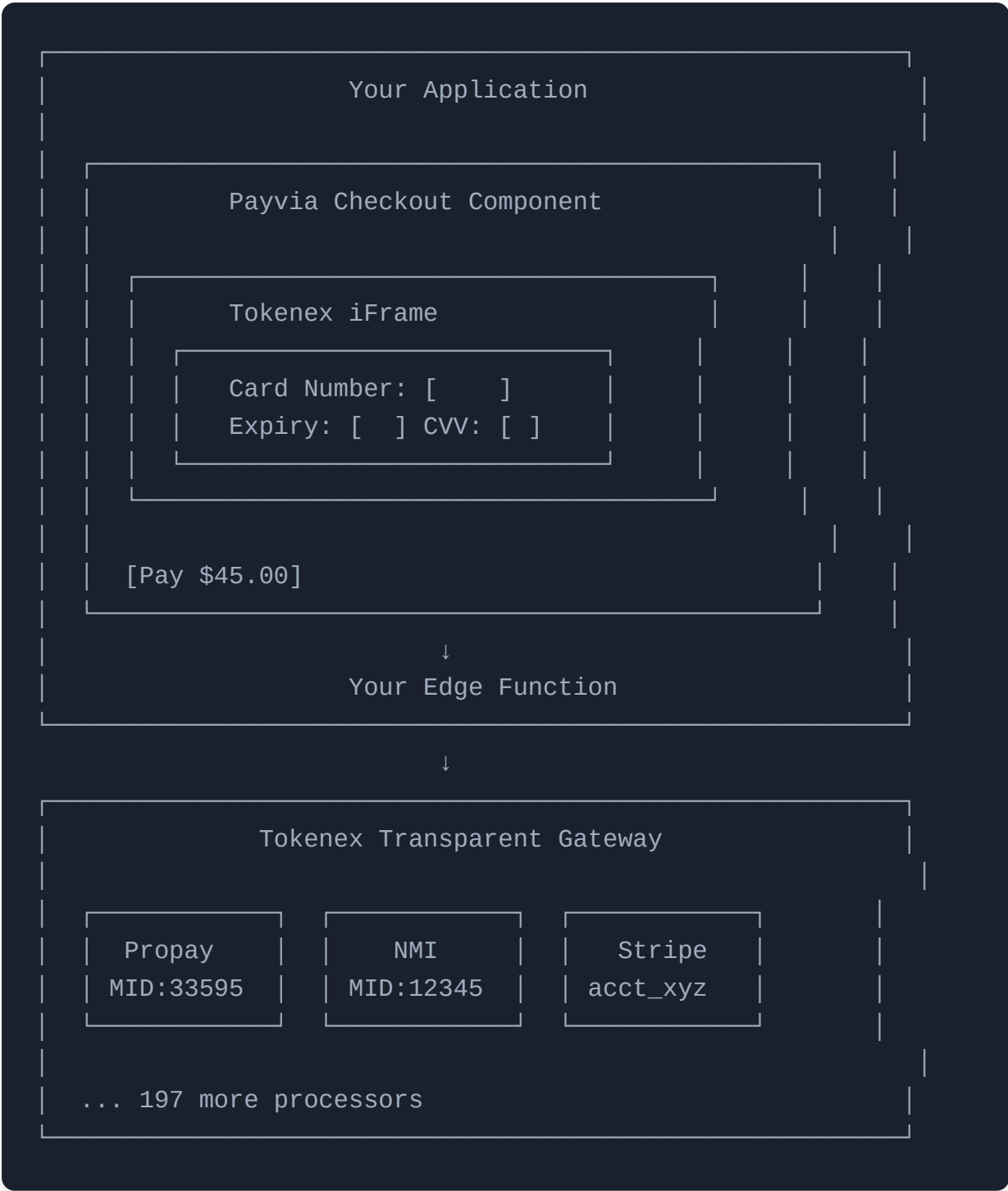
Payvia Wrapper Implementation Details

What is Payvia?

Payvia is a processor-agnostic embedded checkout solution that wraps Tokenex's iFrame to provide:

- Unified checkout experience across 200+ processors
- PCI-compliant payment collection
- Multi-processor routing capabilities
- Consistent API regardless of backend processor

Architecture



Component Breakdown

1. Payvia Checkout Modal

```
// React component wrapping Tokenex iFrame
interface PayviaCheckoutProps {
  amount: number;
  currency: string;
  merchantId: string;
  processor: 'propay' | 'nmi' | 'stripe' | string;
  onSuccess: (transactionId: string) => void;
  onError: (error: Error) => void;
  customerInfo: CustomerInfo;
  eventInfo: EventInfo;
}
```

2. Tokenex iFrame Initialization

```
// Load Tokenex iframe
const tokenExIframe = new TokenEx.Iframe('tokenExIframeDiv', {
  tokenExID: TOKENEX_ID,
  tokenScheme: 'PCI',
  origin: window.location.origin,
  timestamp: Date.now(),
  authenticationKey: generateHMAC(),
  styles: {
    base: 'font-family: Arial; padding: 10px;',
    focus: 'border: 2px solid blue;',
    error: 'border: 2px solid red;',
  },
  pci: true,
  cvv: true,
  cvvContainerID: 'cvv-container',
  enablePrettyFormat: true,
  placeholder: '0000 0000 0000 0000',
});

// Listen for tokenization
tokenExIframe.on('tokenize', (data) => {
  const token = data.token;
  processPayment(token);
});

// Load the iframe
tokenExIframe.load();
```

3. HMAC Authentication

```
// Generate authentication key for iframe
function generateHMAC(
  tokenExID: string,
  apiKey: string,
  timestamp: number
): string {
  const crypto = require('crypto');
  const message = `${tokenExID}|${timestamp}`;
  return crypto
    .createHmac('sha256', apiKey)
    .update(message)
    .digest('hex');
}
```

4. Payment Processing Flow

```
async function processPayment(token: string) {
  try {
    // 1. Collect device data
    const deviceData = {
      ipAddress: await getClientIP(),
      userAgent: navigator.userAgent,
      browserLanguage: navigator.language,
      screenResolution: `${screen.width}x${screen.height}`,
      timezone: Intl.DateTimeFormat().resolvedOptions().timeZone,
    };

    // 2. Call your edge function
    const response = await fetch(
      `${SUPABASE_URL}/functions/v1/payvia-process`,
      {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
          'Authorization': `Bearer ${SUPABASE_ANON_KEY}`,
        },
        body: JSON.stringify({
          tokenexToken: token,
          amount: 45.00,
          currency: 'USD',
          orderId: generateOrderId(),
          merchantId: '33595002',
          processor: 'propay',
          gatewayType: 'digitzs',
          customerInfo,
          eventInfo,
          deviceData,
        }),
      }
    );

    const result = await response.json();

    // 3. Handle response
    if (result.success) {
```

```
        onSuccess(result.transactionId);
        await createTicketSocketOrder(result);
    } else {
        onError(new Error(result.message));
    }
} catch (error) {
    onError(error);
}
}
```

5. Edge Function (Tokenex Integration)

```
// supabase/functions/payvia-process/index.ts
async function processTokenexTransaction(data: TransactionRequest) {
  // Call Tokenex Transparent Gateway API
  const response = await fetch(
    'https://api.tokenex.com/v2/TransparentGatewayAPI',
    {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'tx-tokenex-id': TOKENEX_ID,
        'tx-apikey': TOKENEX_API_KEY,
      },
      body: JSON.stringify({
        Token: data.tokenexToken,
        Amount: data.amount.toFixed(2),
        Currency: data.currency,
        Gateway: 'NMI', // or 'Propay', 'Stripe', etc.
        GatewayCredentials: {
          MerchantID: data.merchantId,
          // Additional credentials if needed
        },
        TransactionType: 'sale',
        OrderID: data.orderId,
        CustomerInfo: {
          FirstName: data.customerInfo.firstName,
          LastName: data.customerInfo.lastName,
          Email: data.customerInfo.email,
          Phone: data.customerInfo.phone,
        },
        BillingInfo: {
          // Address fields
        },
        DeviceData: data.deviceData,
      }),
    },
  );
};
```



```
    return await response.json();  
  }
```

Processor Configuration

Propay MID 33595002

```
const propayConfig = {  
  processor: 'propay',  
  merchantId: '33595002',  
  gateway: 'NMI', // Digitzs is NMI white label  
  credentials: {  
    // Stored in Tokenex dashboard  
    merchantId: '33595002',  
  },  
};
```

Multiple Processors

```
const processorConfigs = {
  lightsfest: {
    processor: 'propay',
    merchantId: '33595002',
    gateway: 'NMI',
  },
  dinosaurisland: {
    processor: 'propay',
    merchantId: '33595002',
    gateway: 'NMI',
  },
  // Future events can use different processors
  futureEvent: {
    processor: 'stripe',
    merchantId: 'acct_stripe123',
    gateway: 'Stripe',
  },
};
```

Processor Routing Logic

Dynamic Processor Selection

```
function selectProcessor(eventId: string): ProcessorConfig {
  const event = await getEventDetails(eventId);

  // Route based on event configuration
  if (event.processor_override) {
    return event.processor_override;
  }

  // Route based on amount
  if (event.amount > 1000) {
    return processorConfigs.highVolume;
  }

  // Route based on card type
  if (cardType === 'amex') {
    return processorConfigs.amexProcessor;
  }

  // Default to Propay
  return processorConfigs.lightsfest;
}
```

Styling Payvia Checkout

Custom iFrame Styles

```
const customStyles = {
  base: `
    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", s
    font-size: 16px;
    color: #1a1a1a;
    padding: 12px;
    border: 1px solid #d1d5db;
    border-radius: 8px;
    background: white;
  `,
  focus: `
    border-color: #2563eb;
    outline: none;
    box-shadow: 0 0 0 3px rgba(37, 99, 235, 0.1);
  `,
  error: `
    border-color: #dc2626;
    color: #dc2626;
  `,
  valid: `
    border-color: #16a34a;
  `,
  cardIcon: `
    width: 32px;
    height: 20px;
  `,
  cvvIcon: `
    width: 24px;
    height: 16px;
  `,
};
```

Modal Styling

```
.payvia-modal {  
  position: fixed;  
  top: 50%;  
  left: 50%;  
  transform: translate(-50%, -50%);  
  background: white;  
  border-radius: 16px;  
  box-shadow: 0 20px 60px rgba(0, 0, 0, 0.3);  
  padding: 32px;  
  max-width: 480px;  
  width: 90%;  
  z-index: 1000;  
}  
  
.payvia-overlay {  
  position: fixed;  
  inset: 0;  
  background: rgba(0, 0, 0, 0.5);  
  z-index: 999;  
}
```

Error Handling

Tokenex Errors

```
const tokenexErrors = {
  'Invalid Token': 'Card information is invalid. Please check and',
  'Expired Token': 'Session expired. Please refresh and try again',
  'Gateway Timeout': 'Payment processor is unavailable. Please tr',
  'Insufficient Funds': 'Card has insufficient funds.',
  'Card Declined': 'Card was declined. Please try a different car',
  'Invalid CVV': 'Security code is incorrect.',
  'Invalid Expiration': 'Expiration date is invalid.',
};
```

Graceful Degradation

```
// Fallback to direct integration if Tokenex unavailable
async function attemptPayment() {
  try {
    // Try Tokenex first
    return await processWithTokenex();
  } catch (error) {
    if (error.code === 'TOKENEX_UNAVAILABLE') {
      console.warn('Tokenex unavailable, using direct integration');
      return await processWithDigitzsDirect();
    }
    throw error;
  }
}
```

Testing Payvia Wrapper

Test Mode Toggle

```
const PAYVIA_TEST_MODE = import.meta.env.VITE_PAYVIA_TEST_MODE === 'true'

if (PAYVIA_TEST_MODE) {
  // Use Tokenex sandbox
  TOKENEX_ID = 'test_tokenex_id';
  TOKENEX_API_KEY = 'test_api_key';

  // Use test MID
  MERCHANT_ID = 'test_merchant_id';
}
```

Test Scenarios

```
// Test successful payment
await testPayment({
  card: '4111111111111111',
  expected: 'success',
});

// Test declined payment
await testPayment({
  card: '4000300011112220',
  expected: 'declined',
});

// Test insufficient funds
await testPayment({
  card: '40000000000000002',
  expected: 'insufficient_funds',
});

// Test network error
await testPayment({
  card: '4111111111111111',
  simulateNetworkError: true,
  expected: 'network_error',
});
```


Webhook Integration

Tokenex Webhooks

```
// supabase/functions/tokenex-webhook/index.ts
Deno.serve(async (req) => {
  const signature = req.headers.get('tx-signature');
  const body = await req.text();

  // Verify signature
  if (!verifyWebhookSignature(body, signature)) {
    return new Response('Invalid signature', { status: 401 });
  }

  const event = JSON.parse(body);

  switch (event.type) {
    case 'transaction.success':
      await handleSuccessfulTransaction(event.data);
      break;
    case 'transaction.failed':
      await handleFailedTransaction(event.data);
      break;
    case 'chargeback.created':
      await handleChargeback(event.data);
      break;
  }

  return new Response('OK');
});
```

Advantages of Payvia Wrapper

1. **Processor Agnostic:** Switch processors without changing frontend code
2. **PCI Compliant:** Tokenex handles card data securely

3. **200+ Processors:** Access to vast processor network
4. **Consistent API:** Same interface regardless of backend
5. **Reduced Risk:** Tokenex manages processor relationships
6. **Easy Testing:** Sandbox mode for all processors
7. **Advanced Routing:** Route based on card type, amount, etc.
8. **Fraud Tools:** Built-in fraud detection and 3D Secure

Disadvantages / Considerations

1. **Cost:** Monthly Tokenex fees + processor fees
2. **Dependency:** Relies on Tokenex uptime
3. **Complexity:** Additional layer in payment stack
4. **Debugging:** Harder to troubleshoot with extra hop
5. **Billing Issues:** As experienced with recent deactivation

Recovery Plan Summary

1. **Immediate:** Use direct Digitzs integration (implemented)
2. **Short Term:** Resolve Tokenex billing issue
3. **Long Term:** Maintain both integrations with feature flag
4. **Future:** Consider building own multi-processor router

Next Steps When Tokenex Reactivates

1. Verify account access
2. Check MID configurations still exist
3. Test in sandbox environment

4. Update API keys if changed
5. Test end-to-end flow
6. Monitor first transactions closely
7. Keep direct integration as backup